

Examen réparti 2 DataCloud Master 2 SAR

Jonathan Lejeune

Février 2024

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur $21\frac{1}{2}$ points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Lors de la correction il sera tenu compte de la simplicité et de la lisibilité de vos réponses
- S'abstenir de répondre est préférable à répondre une absurdité
- Vous trouverez en annexe de la documentation ou des rappels sur les API manipulées

Exercice 1 – Blockchain (4 points)

Merci de répondre directement sur le polycopié dédié à cette partie. N'oubliez pas d'y indiquer votre nom et votre prénom.

Exercice 2 – Questions de cours (5 points)

Cet exercice est fait pour évaluer la compréhension des différents concepts et mécanismes vus en cours. **Soyez concis et synthétique.**

Question 1 1 point

Un client C souhaite accéder à une ligne L d'une table Hbase T . On suppose que C a un cache vide et n'a pas encore initié de requête. Quelles sont les interactions réseaux pour obtenir le contenu de la ligne L sur le client C ? Vous préciserez pour chaque interaction son intérêt et quel service elle concerne.

Question 2 $1\frac{1}{2}$ points

Soit une table Hbase T vide ne contenant qu'une seule column family. Soient 3 clients Hbase concurrents (2 écrivains $E1$ et $E2$, 1 lecteur L) envoyant chacun une requête sur la rowkey A de la table T :

- $E1$ fait `put(A, (x=3,z=6))`
- $E2$ fait `put(A, (x=1,y=5))`
- L fait un `get(A, (x,y,z))`

x , y et z étant des colonnes logiques. Pour chaque configuration suivante dire **en justifiant** si il est possible qu'elle soit lue par L :

- $x=1, y=\emptyset, z=6$
- $x=3, y=5, z=6$
- $x=\emptyset, y=5, z=\emptyset$

Question 3 $\frac{1}{2}$ point

Qu'est-ce qui permet à Cassandra de faire un compromis entre AP et CP?

Question 4 1 point

Quelle(s) conséquence(s) a la dénormalisation

- sur les accès en lecture ?
- sur les accès en écriture ?

Question 5 1/2 point

Qu'est qu'une DHT et comment cela fonctionne ?

Question 6 1/2 point

Énoncer le théorème CAP de Brewer.

Exercice 3 – Zookeeper (3 points)

Question 1 1 1/2 points

Rappeler comment implanter un mutex avec Zookeeper. Vous préciserez :

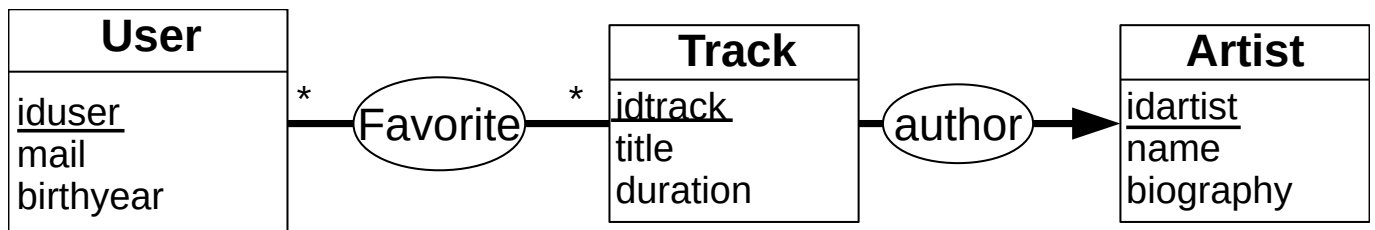
- l'organisation de la structure znodes nécessaires (hiérarchie, contenus, caractère éphémère et/ou séquentiel)
- le pseudo-code/l'algorithme des primitives lock et unlock

Question 2 1 1/2 points

Quelle(s) modification(s) faut-il apporter afin de généraliser ce mutex à un sémaphore ? Pour rappel un sémaphore permet qu'il y ait au plus k clients en section critique (un mutex est un sémaphore où $k = 1$)

Exercice 4 – Musique en ligne (9,5 points)

DixHeures est un fournisseur de contenu musical en ligne. Il permet à des utilisateurs inscrits d'écouter en direct des musiques. Chaque utilisateur peut également configurer une liste de musiques favorites. Le SGBD utilisé pour stocker ces données est Cassandra. Le schéma de données que l'on considère est le suivant :



Les commandes CQL qui ont été lancées pour générer le schéma de données sont les suivantes

```

1 CREATE KEYSPACE IF NOT EXISTS dixheures WITH REPLICATION = { 'class' : 'SimpleStrategy
  ', 'replication_factor': 5 };
2 USE dixheures;
3 CREATE TABLE User (iduser VARCHAR, mail VARCHAR, birthyear INT, PRIMARY KEY (iduser));
4 CREATE TABLE Artist (idartist VARCHAR, name VARCHAR, biography VARCHAR, PRIMARY KEY (
  idartist));
5
6 CREATE TABLE Track (idtrack VARCHAR, title VARCHAR, duration INT, author VARCHAR,
  PRIMARY KEY (idtrack));
7
8 CREATE TABLE Favorite (iduser VARCHAR, idtrack VARCHAR, PRIMARY KEY (iduser, idtrack));
    
```

Question 1 1 point

En ouvrant un shell cql client, on tape la commande `CONSISTENCY QUORUM`; Qu'est-ce que cela implique :

- sur les prochaines commandes `INSERT` ?
- sur les prochaines commandes `SELECT` ?

Soit le code scala suivant :

```

1 object Dixheures extends App {
2   val sc = new SparkContext(new SparkConf().setAppName("DixHeures")
3                               .setMaster(this.args(0))
4                               .set("spark.cassandra.connection.host", this.args(1))
5                               )
6 }

```

Question 2 1 point

Complétez ce code par les méthodes `loadUsers`, `loadArtists`, `loadTracks` et `loadFavorites` qui renvoient un RDD contenant les éléments de la table correspondante.

Question 3 1 point

Ajoutez la méthode `addFavorite` qui permet d'ajouter dans la base de données un nouveau favori passé en paramètre.

Question 4 2 points

Définir une fonction qui calcule pour chaque nom d'artiste l'âge moyen de ses auditeurs. La méthode devra donc renvoyer un résultat de type `RDD[(String,Double)]`. L'âge d'un auditeur est pondéré par le nombre de fois où on peut trouver une musique de l'artiste dans sa liste de favoris. Exemple : Considérons les artistes Vivaldi et Eminem et les utilisateurs Arlette, Bernard et Cyprien nés respectivement en 1939 (85 ans), 1962 (62) et 2004 (20 ans). Arlette a dans ses favoris 10 musiques de Eminem et 1 de Vivaldi, Bernard 2 musiques de Eminem et 1 musiques de Vivaldi, et Cyprien 0 musiques de Eminem et 6 musiques de Vivaldi. Ce qui donne ainsi pour Vivaldi $(1*85 + 1*62 + 6*20)/(1+1+6) = 33,37$ ans et pour Eminem : $(10*85 + 2*62 + 0*20)/(10+2+0) = 81,16$ ans

N.B. :

- vous pouvez récupérer l'année courante avec `LocalDate.now().getYear`
- Il est conseillé de typer vos variables et de commenter ce que fait chaque ligne de code afin d'en faciliter la correction

Question 5 1½ points

Décrivez une solution pour améliorer les performances du calcul précédent. Vous indiquerez les éventuelles modifications à faire sur le schéma et sur les méthodes codées jusqu'à présent. Il n'est pas attendu que vous donniez du code, mais une explication textuelle **claire, précise et justifiée** de votre démarche.

On prend en compte désormais le fait que les utilisateurs écoutent des musiques en direct via un lecteur multimédia en ligne. À chaque fois qu'un utilisateur stoppe le lecteur ou change de musique à écouter, une ligne de log est produite. Une ligne de log indique pour un id de musiques et un id d'utilisateur donnés une date de début d'écoute et une date de fin d'écoute. Ces quatre informations sont au format texte et sont séparées par un espace. Les dates sont des estampilles temporelles en millisecondes. Par exemple, la ligne `user25 track78 1580476418419 1580476524440` indique que l'utilisateur d'id `user25` a commencé d'écouter la musique d'id `track78` à la date `1580476418419` et à arrêté l'écoute à la date `1580476524440`. Ces logs sont consultables depuis un flux temps-réel émis par le serveur *listenstat.dixheures.com* sur le port 532.

Question 6 1½ points

Nous souhaitons créer une table Hbase nommée `ListeningStat` qui maintient pour chaque couple (`idtrack`, `iduser`) la durée totale cumulée d'écoute. À terme nous souhaiterons pouvoir calculer le classement des musiques de la plus écoutée à la moins écoutée en interrogeant cette table. Expliquez comment vous organiseriez cette table en indiquant la sémantique des `rowkey`, des `columnFamily`, des `columns` et des `values`. Justifiez vos choix, l'objectif étant bien sûr que l'organisation logique soit la plus adaptée possible à la requête tout en optimisant l'utilisation de la mémoire disque.

Question 7 1½ points

Compléter le programme spark suivant qui de mettre à jour toutes les 5 minutes la table `ListeningStat`. Vous assurerez que la table reste cohérente après la mise à jour.

```

1  val conf = new SparkConf().setAppName("Stream Listening")
2  val sc = new SparkContext(conf)
3  val ssc = new StreamingContext(sc, Minutes(5))
4
5  val stream1 = ssc.socketTextStream("listenstat.dixheures.com", 532)
6  val stream2 = stream1.map(_.split(" "))
7
8  //TODO
9
10 ssc.start()
11 ssc.awaitTermination(

```

Annexes

```

1  object SparkHbaseConnector {
2      class MySparkContext(sc:SparkContext){
3          def hbaseTableRDD(tablename:String, scan:Scan, quorumzookeeper:String="localhost"
4              ):RDD[(ImmutableBytesWritable,Result)]={
5              val hbaseconf = HBaseConfiguration.create()
6              hbaseconf.set("hbase.zookeeper.quorum", quorumzookeeper);
7              hbaseconf.set(TableInputFormat.INPUT_TABLE, tablename);
8              hbaseconf.set(TableInputFormat.SCAN, TableMapReduceUtil.convertScanToString(
9                  scan));
10             return sc.newAPIHadoopRDD(hbaseconf, classOf[TableInputFormat], classOf[
11                 ImmutableBytesWritable], classOf[Result]);
12         }
13     }
14     implicit def sparkContextToMySparkContext(sc:SparkContext):MySparkContext= new
15         MySparkContext(sc)
16
17
18
19     class RDDHbase[M<:Mutation](rdd:RDD[M]){
20         def saveAsHbaseTable(tablename:String, quorumzookeeper:String="localhost"):Unit={
21             rdd.foreachPartition( it => {
22                 val hbaseconf = HBaseConfiguration.create()
23                 hbaseconf.set("hbase.zookeeper.quorum", quorumzookeeper)
24                 val connection = ConnectionFactory.createConnection(hbaseconf)
25                 val table = connection.getTable(TableName.valueOf(tablename))
26                 val puts = new ArrayList[Put]
27                 val deletes = new ArrayList[Delete]
28                 for(mut <- it){
29                     mut match {
30                         case put:Put => puts.add(put)
31                         case delete:Delete => deletes.add(delete)
32                         case append:Append => table.append(append)
33                         case increment:Increment => table.increment(increment)
34                         case _ => throw new RuntimeException("Bad type of mutation")
35                     }
36                 }
37                 if(!puts.isEmpty){
38                     table.put(puts)
39                 }
40                 if(!deletes.isEmpty()){
41                     table.delete(deletes)
42                 }
43                 table.close()
44                 connection.close()
45             })
46         }
47     }
48     implicit def rddToRDDHbase[M<:Mutation](rdd:RDD[M]): RDDHbase[M] = new RDDHbase(rdd)
49 }

```

Rappel API Spark Cassandra connector

- `def cassandraTable(keyspace:String,table:String):CassandraTableScanRDD[CassandraRow]`
- `def cassandraTable[T](keyspace: String,table: String):CassandraTableScanRDD[T]`
- `def saveToCassandra(keyspace:String, table:String, cols:ColumnSelector = AllColumns)`
- `def saveAsCassandraTable(keyspa:String, table:String, cols:ColumnSelector=AllColumns)`

Rappel des principales méthodes de l'API Java de Hbase

Connection

- `Admin getAdmin()`
- `Table getTable(TableName tableName)`

Table

- `TableName getName()`
- `boolean exists(Get get)`
- `Result get(Get get)`
- `ResultScanner getScanner(Scan scan)//ResultScanner is an iterable of Result objects`
- `void put(Put put)`
- `void put(List<Put> puts)`
- `void delete>Delete delete)`
- `void delete(List<Delete> deletes)`
- `Result increment(Increment increment)//returns values of columns after the increment`

TableName

- `static TableName valueOf(String namespace, String tablequalifier)`
- `String getQualifierAsString()`
- `String getNamespaceAsString()`
- `String getNameAsString()//namespace:qualifier`

Mutation (abstract)

- `byte [] getRow()`

Put

- `Put(byte[] rowkey)`
- `Put(byte[] rowkey, long def_ts)`
- `Put addColumn(byte[] family, byte[] col, byte[] val);`
- `Put addColumn(byte[] family, byte[] col, long ts, byte[] val);`

Delete

- `Delete(byte[] rowkey)`
- `Delete(byte[] rowkey, long def_ts)`
- `Delete addColumns( byte [] family, byte [] qualifier);`
- `Delete addColumns( byte [] family, byte [] qualifier, final long timestamp);`

Append

- `Append(byte[] row)`
- `add(byte [] family, byte [] qualifier, byte [] value))`

Increment

- `Increment(byte[] row)`
- `Increment addColumn(byte[] family, byte[] qualifier, long amount)`

Scan

- `Scan()//scan all the table`
- `Scan(byte[] startRow, byte[] stopRow)`
- `Scan addFamily(byte [] family)`
- `Scan addColumn(byte [] family, byte [] qualifier)`
- `byte[] getStartRow()`
- `byte[] getStopRow()`

Get

- Get(byte [] row)
- Get addFamily(byte [] family)
- Get addColumn(byte [] family, byte [] qualifier)

Result

- byte[] getValue(byte[] family, byte[] qualifier)
- Map<byte[], byte[]> getFamilyMap(byte[] family)

API for SparkContext	Meaning
new SparkContext (config: SparkConf)	Create a SparkContext that loads settings from a Spark Config object describing the application configuration. Any settings in this config overrides the default configs as well as system properties.
new SparkContext (master: String, appName: String, conf: SparkConf)	Alternative constructor that allows setting common Spark properties directly, : Cluster URL to connect to (e.g. mesos://host:port, spark://host:port, local[4]), a name for your application, to display on the cluster web UI and a SparkConf object specifying other Spark parameters
textFile (path: String, minPartitions: Int = defaultMinPartitions): RDD[String]	Read a text file from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI, and return it as an RDD of Strings. Path is the path to the text file on a supported file system, minPartitions is the suggested minimum number of partitions for the resulting RDD. It returns RDD of lines of the text file
sequenceFile [K, V](path: String, keyClass: Class[K], valueClass: Class[V]): RDD[(K, V)]	Get an RDD for a Hadoop SequenceFile with given key and value types. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, keyClass (resp. valueClass) is the Class of the key (resp. the value) associated with SequenceFileInputFormat. It returns RDD of tuples of key and corresponding value.
parallelize [T](seq: Seq[T], numSlices: Int): RDD[T]	Distribute a local Scala collection to form an RDD. NumSlices is number of partitions to divide the collection into. It returns the RDD representing distributed collection
newAPIHadoopFile [K, V, F <: InputFormat[K, V]](path: String, fClass: Class[F], kClass: Class[K], vClass: Class[V], conf: Configuration = hadoopConfiguration): RDD[(K, V)]	Get an RDD for a given Hadoop file with an arbitrary new API InputFormat and extra configuration options to pass to the input format. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, fclass is the storage format of the data to be read , kClass (resp. vClass) is the Class of the key (resp. the value) Hadoop configuration. It returns RDD of tuples of key and corresponding value
broadcast [T](value: T): Broadcast[T]	Broadcast a read-only variable to the cluster, returning a org.apache.spark.broadcast.Broadcast object for reading it in distributed functions. The variable will be sent to each cluster only once.
collectionAccumulator [T]: CollectionAccumulator[T]	Create and register a CollectionAccumulator, which starts with empty list and accumulates inputs by adding them into the list.
doubleAccumulator : DoubleAccumulator longAccumulator : LongAccumulator	Create and register a double or long accumulator, which starts with 0 and accumulates inputs by add.
register (acc: AccumulatorV2[_, _]): Unit	Register the given accumulator. Accumulators must be registered before use, or it will throw exception.
stop (): Unit	Shut down the SparkContext.

Simple transformation for RDD[T]	Meaning
map [U](f: (T) =>U): RDD[U]	Return a new distributed dataset formed by passing each element of the source through a function f .
mapValues [U](f: V => U): RDD[(K, U)]	Pass each value in the key-value pair RDD through a map function without changing the keys this also retains the original RDD's partitioning.
filter (f: (T) =>Boolean): RDD[T]	Return a new dataset formed by selecting those elements of the source on which f returns true.
flatMap [U](f: (T) =>TraversableOnce[U]): RDD[U]	Similar to map, but each input item can be mapped to 0 or more output items (so f should return a Seq rather than a single item).
flatMapValues [U](f: V => TraversableOnce[U]): RDD[(K, U)]	Pass each value in the key-value pair RDD through a flatMap function without changing the keys ; this also retains the original RDD's partitioning.

union (other: RDD[T]): RDD[T]	Return a new dataset that contains the union of the elements in the source dataset and the argument.
zip [U](other: RDD[U]): RDD[(T, U)]	Zips this RDD with another one, returning key-value pairs with the first element in each RDD, second element in each RDD, etc. <i>Assumes that the two RDDs have the same number of partitions and the same number of elements in each corresponding partition</i> (e.g. one was made through a map on the other).
<u>zipWithIndex</u> (): RDD[(T, Long)]	Zips this RDD with its element indices. The ordering is first based on the partition index and then the ordering of items within each partition. So the first item in the first partition gets index 0, and the last item in the last partition receives the largest index. This is similar to Scala's zipWithIndex but it uses Long instead of Int as the index type. This method needs to trigger a spark job when this RDD contains more than one partitions.
keyBy [K](f: (T) =>K): RDD[(K, T)]	Creates tuples of the elements in this RDD by applying f
glom (): RDD[Array[T]]	Return an RDD created by coalescing all elements within each partition into an array.

Large transformation for RDD[T]	Meaning
intersection (other: RDD[T]): RDD[T] intersection (other: RDD[T], numPartitions: Int): RDD[T] intersection (other: RDD[T], part: Partitioner): RDD[T]	Return the intersection of this RDD and another one. The output will not contain any duplicate elements, even if the input RDDs did.
distinct (): RDD[T] distinct (numPartitions: Int): RDD[T]	Return a new dataset that contains the distinct elements of the source dataset.
cartesian [U](other: RDD[U]): RDD[(T, U)]	When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).
repartition (numPartitions: Int): RDD[T]	Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
subtract (other: RDD[T]): RDD[T] subtract (other: RDD[T], numPartitions: Int): RDD[T] subtract (other: RDD[T], p: Partitioner): RDD[T]	Return an RDD with the elements from <i>this</i> that are not in <i>other</i> .
sortBy [K](f: (T) =>K, ascending: Boolean = true, numPartitions: Int = this.partitions.length): RDD[T]	Return this RDD sorted by the given key function.

Large transformation for RDD[T] and T= (K,V)	Meaning
groupByKey (): RDD[(K, Iterable[V])] groupByKey (numPartitions:Int): RDD[(K, Iterable[V])] groupByKey (p:Partitioner): RDD[(K, Iterable[V])]	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.
reduceByKey (func: (V, V) =>V): RDD[(K, V)] reduceByKey (numPartitions:Int, func: (V, V) =>V): RDD[(K, V)] reduceByKey (partitioner: Partitioner, func: (V, V) =>V): RDD[(K, V)]	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function <i>func</i> , which must be of type (V,V) => V.
sortByKey (ascending: Boolean = true, numPartitions: Int = self.partitions.length) : RDD[(K, V)]	When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean <i>ascending</i> argument.

join [W](other: RDD[(K, W)]): RDD[(K, (V, W))] join [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, W))] join [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, (V, W))]	Return an RDD containing all pairs of elements with matching keys in <i>this</i> and <i>other</i> . Each pair of elements will be returned as a (k, (v1, v2)) tuple, where (k, v1) is in <i>this</i> and (k, v2) is in <i>other</i>
leftOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (V, Option[W]))] leftOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, Option[W]))] leftOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (V, Option[W]))]	Perform a left outer join of <i>this</i> and <i>other</i> . For each element (k, v) in <i>this</i> , the resulting RDD will either contain all pairs (k, (v, Some(w))) for w in <i>other</i> , or the pair (k, (v, None)) if no elements in <i>other</i> have key k.
rightOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (Option[V], W))] rightOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], W))] rightOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], W))]	Perform a right outer join of <i>this</i> and <i>other</i> . For each element (k, w) in <i>other</i> , the resulting RDD will either contain all pairs (k, (Some(v), w)) for v in <i>this</i> , or the pair (k, (None, w)) if no elements in <i>this</i> have key k.
fullOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (Option[V], Option[W]))] fullOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], Option[W]))] fullOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], Option[W]))]	Perform a full outer join of <i>this</i> and <i>other</i> . For each element (k, v) in <i>this</i> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for w in <i>other</i> , or the pair (k, (Some(v), None)) if no elements in <i>other</i> have key k. Similarly, for each element (k, w) in <i>other</i> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for v in <i>this</i> , or the pair (k, (None, Some(w))) if no elements in <i>this</i> have key k.
cogroup [W](o: RDD[(K, W)]): RDD[(K, (Iterable[V], Iterable[W]))] cogroup [W](o: RDD[(K, W)], numPart: Int): RDD[(K, (Iterable[V], Iterable[W]))] cogroup [W](o: RDD[(K, W)], p: Partitioner): RDD[(K, (Iterable[V], Iterable[W]))]	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called groupWith .
subtractByKey [W](other: RDD[(K, W)]): RDD[(K, V)] subtractByKey [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, V)] subtractByKey [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, V)]	Return an RDD with the pairs from <i>this</i> whose keys are not in <i>other</i>

Action for RDD[T]	Meaning
reduce (f: (T, T) =>T): T	Aggregate the elements of the dataset using a function <i>func</i> (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect (): Array[T]	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count ():Long	Return the number of elements in the dataset.
first (): T	Return the first element of the dataset (similar to take(1)).
take (num: Int): Array[T]	Return an array with the first <i>n</i> elements of the dataset.
top (num: Int)(implicit ord: Ordering[T]): Array[T]	Returns the top k (largest) elements from this RDD as defined by the specified implicit Ordering[T] and maintains the ordering.
saveAsObjectFile (path: String): Unit	Save this RDD as a SequenceFile of serialized objects.
saveAsTextFile (path: String): Unit	Save this RDD as a text file, using string representations of elements.
foreach (f: (T) =>Unit): Unit	Run a function <i>func</i> on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems.